FamilyJoy Final Report

Micro-Credentials in Software Development

Certificate in Software Development

MCSD51

Project: FamilyJoy

Author: DongHan WU

Student ID: 737214421

Preface

This report presents the final outcome of the FamilyJoy project. The content is based mainly on the implemented codebase, the planning and design documents produced during development, the black-box testing records from each sprint, and the final submission materials prepared at the end of the project.

FamilyJoy was completed independently by DongHan WU, but the work extended well beyond coding. The development process also included requirements analysis, staged planning, page and workflow design, system implementation, testing and validation, documentation preparation, and final delivery packaging. Accordingly, the transferable-skills discussion is framed through multi-role responsibility within an individual project rather than through team collaboration.

A public repository is included as part of the submission package so that the implementation, supporting documents, and project artefacts can be accessed in one place:

https://github.com/dh-wu23210/MCSD51_DongHan_WU_FamilyJoy_Project

Table of Contents

Preface 2
Table of Contents 3
1. Introduction 6
1.1 Background 6
1.2 Market Context 6
1.3 Project Value 7
1.4 Problem Statement 7
1.5 Project Vision 7
1.6 Stakeholders 7
1.7 Objectives 7
1.8 Scope and Constraints 7
1.9 Ethical 8
1.10 Privacy Considerations 8
2. Initial Proposal Summary 8
3. Changes from the Initial Proposal 9
3.1 Scope Changes 9
3.2 Reasons for Change 9
3.3 Benefits of the Changes 10
4. Consolidated Project Requirements 10
4.1 Functional Requirements 10
4.2 Non-Functional Requirements 12
4.3 Requirements Traceability 13
5. Problem Domain and Use Cases 13
5.1 Problem Domain Sketch 13
5.2 Core Use Cases 14
5.3 Use Case Diagrams 15
6. Final Project Planning 15
6.1 User Stories and Epic Coverage 16
6.2 User Story Format and Screenshot Example 16
6.3 Sprint and Timeline Planning 17

7. SDLC and Project Delivery Process 19
7.1 Development Methodology 19
7.2 Sprint-Based Planning and Execution 19
7.3 Documentation-Driven Development 20
7.4 Iterative Refinement from MVP to Expanded Features 20
7.5 Individual Role Coverage and Transferable Skills 20
7.6 Final Delivery Packaging 20
8. Project Timeline 21
9. Final System Overview 21
9.1 Role Overview 22
9.2 Core Functional Modules 22
9.3 Closed-Loop Product Logic 22
9.4 Role-Based Showcase Mapping 22
10. Design Documents 22
10.1 Architecture Design 23
10.2 Database Design 23
10.3 Data Structures and Core Logic Techniques 24
10.4 Interface and UI Design 24
10.5 Key Workflow Diagrams ..... **Error! Bookmark not defined.**
10.6 Security and Access Design 30
10.7 Non-Functional Design 31
11. Developer Documentation 31
11.1 Directory Structure 31
11.2 Server Layer Responsibilities 31
11.3 Client Layer Responsibilities 32
11.4 Admin Layer Responsibilities 32
11.5 Key Modules to Explain in Detail 32
11.6 Development Workflow 32
12. Tools and Technologies Used 32
13. Deliverables and Software Overview 34
14. Testing and Validation 35
14.1 Testing Strategy 35

14.2 Core Test Areas 35
14.3 Test Results 36
15. User Guide 40
15.1 Parent Guide 40
15.2 Child Guide 40
15.3 Admin Guide 40
16. Configuration and Deployment Guide 41
17. Future Enhancements 41
18. Conclusion 41
References 42
External Sources 42
Internal Project Documents 42
Appendix 43
Appendix A. Initial Proposal 43
Appendix B. Project Planning Artifacts 43
Appendix C. PRD Evidence by Sprint 44
Appendix D. Technical Design and Database Evidence 45
Appendix E. Black-Box Testing Evidence 45
Appendix F. Screenshots and Figure Assets 46
Appendix G. Version Control and Submission Evidence 47

1. Introduction

FamilyJoy was created to address a common family problem that appears frequently but is rarely handled in a structured way: parents know what children are expected to do each day and continue to remind them, but whether tasks were completed, to what extent they were completed, and whether rewards were actually delivered often depend on temporary verbal communication. Once the number of tasks increases, that approach becomes unreliable.

In everyday family life, task assignment, completion confirmation, and reward fulfilment are usually connected. If these stages are not recorded clearly, repeated reminders, inconsistent understanding, and unfair outcomes become common. FamilyJoy was therefore never intended to be only a child-friendly to-do list. The goal was to connect these stages into a clear, visible, and traceable workflow.

The final project became a role-based family task system. Parents create and assign tasks, children complete and submit them, and the system records state and results, then extends those results through rewards, backpack items, wishes, and Spirit Tree interaction. In addition, the system includes a separate admin module for governance and system-level visibility.

1.1 Background

Family routine management does not look complicated on the surface, but the real difficulty lies in repeating it consistently over time. Parents need to explain expectations clearly and also follow up on results. Children need to know what they are expected to do, what has already been done, and what action is still required after completion. If all of this depends only on verbal communication, the workflow easily breaks down.

Traditional approaches usually rely on paper notes, spoken reminders, or parents simply remembering everything themselves. These methods may work when tasks are few and the family is small, but they become difficult to sustain when tasks increase, more family members are added, and reward rules become more complex. The idea behind FamilyJoy was to turn these scattered actions into a clearer digital process.

1.2 Market Context

FamilyJoy is closer to an internal family coordination tool than to a generic task-management application. It focuses on how family roles cooperate around tasks. Parents need a lower-cost way to organise daily routines, while children need more visible progress and clearer feedback.

So the project does not belong to a generic productivity scenario. Instead, it addresses several very specific family-management problems: task ownership is often unclear, completion status lacks a shared record, the review process is easily blurred, and reward fulfilment is difficult to keep consistent over time. The design of FamilyJoy was built gradually around these connected issues.

1.3 Project Value

If a system can reduce the cost of repeated reminders and repeated confirmation inside a family, while also allowing children to see clearer results from their actions, then it has practical value. For parents, daily management becomes more efficient. For children, tasks are no longer only passive obligations; they can also see how their behaviour affects later outcomes.

Children are already familiar with digital interaction. Compared with simply writing chores as plain text items, a system that combines visible progress, reward feedback, and a more approachable interface is more likely to be used consistently instead of being abandoned quickly.

1.4 Problem Statement

FamilyJoy mainly addresses three related problems. First, the manual follow-up cost for parents is too high because they must keep reminding and re-checking progress. Second, there is no shared status between task assignment and review, so completion is often unclear. Third, reward promises are hard to maintain consistently over time, which affects both fairness and motivation.

1.5 Project Vision

The project aimed to establish a family task loop that is structured, role-based, and relatively child-friendly. In the delivered design, parents control task assignment and review, children receive clear feedback and visible progress, and the system connects tasks, rewards, and family routine management into one coherent workflow.

1.6 Stakeholders

The main stakeholders in this project are parent users, child users, and admin users. Parents organise and review tasks. Children carry out tasks and directly receive reward feedback. Admin users do not participate in the family task workflow itself, but instead provide governance functions such as the dashboard, users, families, and logs.

1.7 Objectives

The final project objectives can be summarised as follows: deliver a working role-based family task system; keep the family-facing side and the admin side clearly separated in terms of access; implement task processing based on state transitions; and produce complete planning, design, testing, and delivery documentation. In addition, the project aimed to demonstrate disciplined software development through sprint planning, requirements traceability, and version-control practice.

1.8 Scope and Constraints

This was an individual project, and that directly meant the scope and technical choices could not expand without limit. In order to deliver a system that could run stably and also be explained

clearly within the available time, the project remained within a server-side rendered web architecture rather than expanding into a more complex SPA. The reason for keeping SSR was not only technical preference. It was better suited to complexity control in the current project and also preserved room for later extension into data analysis. At the current stage, no further development was carried out in that direction, so there was no reason to introduce a higher architectural cost only for a different front-end interaction style.

An MVP-first approach was used throughout development. The first priority was to make the family task loop itself work end to end, and only then to add reward circulation, backpack and wish handling, Spirit Tree support, calendar support, mailbox visibility, and admin governance. That progression was better suited to an individual project and also helped prevent the project from losing focus midway through development. From my point of view, protecting the core family loop first and only then extending reward, wish, and admin modules was more reliable than trying to unfold every idea at the beginning, because the stability of the main system directly affects every later feature.

1.9 Ethical

Because the system is child-facing and also involves household data controlled by parents, ethical issues cannot be ignored. The project could not treat gamification as a pure behaviour-pressure tool. That might make the system look more effective, but not necessarily healthier. Rewards in the system are therefore positioned as a structured feedback mechanism rather than as an unrestricted control method.

The project also tried to avoid making interface language and feature settings too narrow. For example, the Spirit Tree feature carries symbolic meaning, but in the product it is used more as an expression of growth than as a forced cultural symbol with one fixed interpretation. The overall system tries to stay neutral, readable, and broadly inclusive of different family structures.

1.10 Privacy Considerations

Privacy is mainly handled through family-bounded data access, role-separated routing, session control, and a clear distinction between the family-facing side and the admin side. Most business data is limited to the relevant family boundary, and the admin module is intentionally separated from the normal user entry path so that governance authority does not become mixed with normal family usage.

2. Initial Proposal Summary

The original proposal described FamilyJoy as a digital system for family routine management, with the goal of improving daily family coordination through clearer task assignment, shared

visibility, and reward-related feedback. Even at proposal stage, the project was already focused less on general productivity and more on specific family interaction.

The target users at that stage were mainly parents and children. Parents were expected to establish rules and review results, while children were expected to carry out tasks and receive feedback. The basic assumption behind the proposal was that if the original routine process, which depended heavily on spoken reminders, could be made clearer and more visible, then family friction could be reduced and consistency of habit execution could improve.

The original scope mainly included account creation, family management, task assignment, completion visibility, and a reward mechanism. The proposal also assumed that the system would be developed gradually through staged iterations rather than built all at once. The retained original proposal is referenced in Appendix A.

3. Changes from the Initial Proposal

The final delivered system remains broadly consistent with the original proposal in terms of problem definition and target users. However, during implementation the system gradually narrowed from a broad family chores tool into a more explicit closed-loop task system. The project did not drift away from the original direction; rather, development made the truly important flows more concrete.

3.1 Scope Changes

The most obvious change is that the task workflow became much more clearly defined. In the final system, a quest is not simply created and checked off after completion. It moves through assignment, child submission, parent review, and reward-result update. This lifecycle eventually became the core of the entire system.

At the same time, the project expanded into areas that were mentioned in the proposal but not yet concrete at that stage, such as backpack items, wishes, Spirit Tree interaction, calendar support, mailbox visibility, admin governance, login-event records, and a more complete PRD, TDD, and BTD documentation set. On the other hand, some technical boundaries were deliberately retained. The system remained SSR-based instead of changing to SPA, and stable, maintainable solutions were prioritised over heavier expansion paths.

3.2 Reasons for Change

These changes were not made simply to make the project look larger. They emerged naturally during implementation. Some changes came from the practical limits of an individual project: time was limited, so implementation complexity had to be controlled. Other changes came from

the internal consistency requirements of the product itself: if role boundaries, state transitions, and reward outcomes remained vague, the system would not stabilise properly.

These changes mainly came from complexity control, priority adjustment in scheduling, and changes in the development cycle. As the codebase grew, issues such as access control, first-login password change, review logic, and reward circulation could no longer remain vague. Defining them more clearly made the system more predictable, testable, and better aligned with the implemented result described in the report.

3.3 Benefits of the Changes

These adjustments ultimately made the project more solid. The real value of the system does not lie in the number of features, but in whether the most important workflow is actually connected. The current version is stronger because it can point to real implementation evidence instead of remaining at conceptual level.

In practice, this combination of narrowing some areas and expanding others also improved maintainability. A system with clear task states and clear role boundaries is easier to explain, easier to test, and easier to validate. That also matters for the credibility of the final report.

4. Consolidated Project Requirements

The final requirements do not come from one single document. They were consolidated from PRD documents across different sprints, user stories, planning files, black-box testing summaries, and the implemented behaviour of the final system. The purpose of this section is to turn those distributed materials into a project-level requirement baseline that is clear enough for assessment and traceable enough for technical explanation.

4.1 Functional Requirements

ID	Functional Requirement	Primary Role	Status
FR-01	The system shall allow creation of a family account and family identity.	Parent	Implemented
FR-02	The system shall allow creation of parent and child member accounts within a family.	Parent	Implemented
FR-03	The system shall support login, logout, and forced first-	Parent / Child	Implemented

	password change.		
FR-04	The system shall allow parent users to create and manage quest templates.	Parent	Implemented
FR-05	The system shall allow parent users to assign quests to children by date.	Parent	Implemented
FR-06	The system shall allow child users to view quests and submit completion.	Child	Implemented
FR-07	The system shall allow parent users to approve or reject submitted work.	Parent	Implemented
FR-08	The system shall support reward, crystal, and shop interaction.	Parent / Child	Implemented
FR-09	The system shall support backpack items, wishes, and spirit tree interaction.	Parent / Child	Implemented
FR-10	The system shall provide calendar and mailbox visibility for family activity.	Parent / Child	Implemented
FR-11	The system shall enforce role separation and family-bound access.	All roles	Implemented
FR-12	The system shall provide a separate admin portal with dashboard, user, family, and log views.	Admin	Implemented

The delivered functional baseline includes account registration and login, initial password change, family creation, member creation and update, quest template creation, quest assignment, child quest viewing, child submission, parent review, reward and crystal handling, shop interaction, backpack visibility, wish creation, spirit tree display, calendar history, mailbox visibility, and separate admin access for dashboard, users, families, and logs. These functions represent the minimum complete workflow that the final project had to preserve.

Together, these functional requirements define a closed loop: the parent creates and manages, the child acts and submits, the system records and updates states, and the result becomes visible again through rewards, history, and growth-oriented feedback.

4.2 Non-Functional Requirements

The non-functional baseline includes role separation, family-bound access control, maintainable server-side layering, relational data consistency, session-based authentication, clear SSR page rendering, usable response times for normal local development, and sufficient documentation to support explanation, validation, and handover. In addition, the project had to remain achievable within the scope of an individual development process rather than assuming team-scale implementation capacity.

These non-functional requirements matter because FamilyJoy is not judged only by whether pages exist, but also by whether the workflow remains understandable, secure enough for its scope, and maintainable across multiple modules and development stages.

ID	Non-Functional Requirement	Rationale	Evidence
NFR-01	The system shall remain maintainable through a layered server structure.	Clear separation of responsibility reduces change risk.	familyjoy_server structure
NFR-02	The system shall enforce role-separated access.	Prevents misuse and protects workflow boundaries.	Session logic and access guards
NFR-03	The system shall remain usable for both parent and child roles.	Different users need different interface emphasis.	Role-based page groups and dashboards
NFR-04	The system shall preserve data consistency across quest and reward workflows.	State accuracy is needed for fairness.	Relational schema and service logic
NFR-05	The system shall provide safe default behaviour for authentication.	Security and account hygiene are necessary.	Forced password change and session handling
NFR-06	The system shall remain extensible for future analytics and interaction features.	The project should support later growth.	Modular services and schema structure
NFR-07	The system shall remain within the approved implementation	Scope control mattered in an individual project.	Node.js, Express, EJS, MySQL, Vanilla JS

	boundary.		
NFR-08	The project shall produce traceable documentation and validation evidence.	The assessment requires evidence portfolio quality.	PRD, TDD, BTD, planning files

4.3 Requirements Traceability

Requirements were traced through three connected evidence layers. User stories and PRD documents described intended behaviour. The codebase provided implementation evidence through routes, services, views, middleware, and schema structure. Sprint-level BTD records provided validation evidence at the level of feature groups. This created a practical chain from requirement to implementation to validation.

Because of this traceability chain, the final report does not rely on claims alone. Each major requirement can be connected back to planning material, forward to implementation structure, and then further to testing summaries and screenshots.

5. Problem Domain and Use Cases

The problem FamilyJoy addresses is not just task recording. It is the entire set of relationships inside a family around task assignment, execution, review, and reward feedback. Family is the top ownership boundary. Inside that boundary, parent users organise and review, child users execute and submit, while admin sits outside the family workflow and handles system-level visibility and governance.

The project therefore deals with tasks, families, users, quests, review states, crystals, rewards, backpack items, wishes, spirit tree records, calendar history, mailbox messages, and admin logs. These concepts appear both in the database structure and in the organisation of pages and business flows.

5.1 Problem Domain Sketch

From a problem-domain perspective, family is the root node. Parent and child accounts exist within that boundary. Parents define tasks and expectations, children complete and submit tasks, and the system carries the review result into rewards, shopping, wishes, and related later behaviour. Because of this, FamilyJoy behaves more like a closed-loop system for family daily coordination than a collection of unrelated feature pages.

The problem domain sketch therefore emphasises family as the top-level boundary first, then parent and child as the two core actors, quest and reward as the main business entities, and finally supporting domains such as backpack, wish, spirit tree, calendar, and mailbox. Admin

remains outside this household loop because it belongs to governance rather than family interaction.

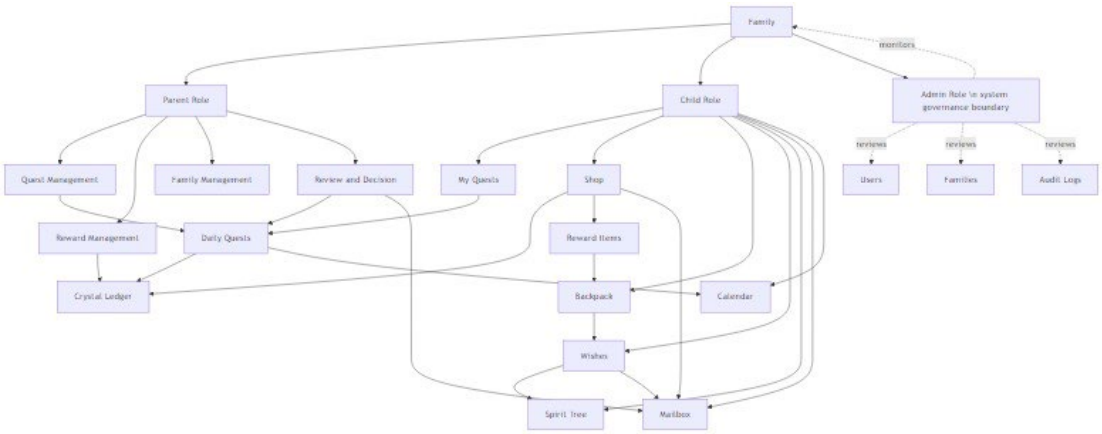


Figure 5-1. FamilyJoy problem domain sketch

5.2 Core Use Cases

The core use cases can be summarised as follows: register or log in to the system, create and manage family members, create quest templates, assign quests, view assigned quests, submit completion, review submissions, manage rewards, use the shop, view backpack items, create wishes, review spirit tree progress, and view calendar and mailbox history. These use cases reflect the actual user journey of the delivered system rather than an abstract UML exercise.

Use Case	Primary Actor	Goal	Outcome
Register family account	Parent	Create a new family workspace	Family and first account are created
Create member account	Parent	Add a new parent or child user	Member account becomes available
Login and change password	Parent / Child	Enter the system securely	Session is established and password state is updated
Create quest template	Parent	Define reusable tasks	Quest template is stored
Assign quest	Parent	Assign a dated task to a child	Daily quest record is created
Submit quest completion	Child	Submit finished work for review	Submission state is recorded
Review quest submission	Parent	Make an approval or rejection decision	Final state and reward result are updated
Manage rewards and shop	Parent / Child	Create and consume reward options	Reward interaction becomes active
Create wish from backpack item	Child	Turn an owned item into a wish	Wish record is created

Review wishes and spirit tree state	Parent	Make a decision and update progression	Related status is updated
Access admin governance pages	Admin	View system-level operating records	Dashboard, users, families, and logs are visible

5.3 Use Case Diagrams

The main use case diagram focuses on the household workflow between parent and child within the FamilyJoy main system. It shows how family setup, quest handling, submission, review, rewards, wishes, spirit tree updates, and related history views connect into one role-based loop. The admin use case diagram is intentionally separate because admin functions are not part of the family task loop itself.

Taken together, the two diagrams provide a clearer explanation than a single overloaded diagram. They align better with the implemented route structure and with the intentional separation between family-facing pages and admin governance pages.

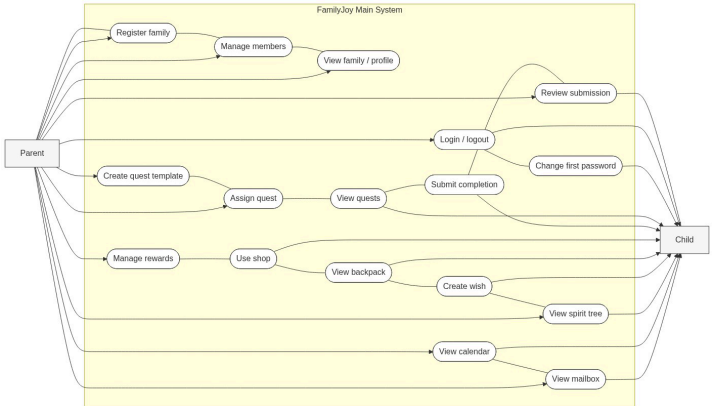


Figure 5-2. FamilyJoy core use case diagram

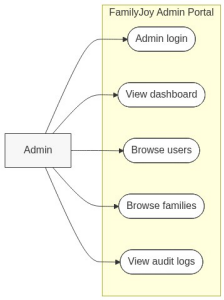


Figure 5-3. FamilyJoy admin use case diagram

6. Final Project Planning

The final project plan is not a static planning table. It is formed jointly by epics, user stories, sprint documents, and timeline files, because FamilyJoy was built, adjusted, and extended across multiple stages rather than completed in one pass.

6.1 User Stories and Epic Coverage

These user stories were broadly organised around login and account handling, family management, quest workflow, reward and shop behaviour, wishes and spirit tree support, calendar support, and administration. The advantage of this grouping is that it follows real user behaviour more closely instead of merely following technical folders.

The key point is not to reproduce every story one by one, but to explain how the epic structure mapped to the delivered system. In the current version, account registration and the task system were protected first because they form the most basic entry point and the core loop of the whole system. By comparison, data-analysis features, fine-grained display features, and enhancement-oriented stories remain valuable, but they do not determine whether the main workflow is viable. They were therefore left to later stages, consistent with the MVP-first approach maintained throughout development.

6.2 User Story Format and Screenshot Example

The user stories in this project follow a consistent planning-record structure rather than being written as loose notes. Each story is recorded in a way that allows planning, sprint assignment, specification links, and later validation to be traced back to the same story entry.

A single story record is structured in this order:

Epic: E2 - Family Management

Status: Final (at epic level)

Story Key / Title: E2-US01 - Create family

User Story: As a System, I want to create family, so that I can complete the Onboarding workflow.

Description: This story defines the expected behavior for "Create family" in the current product flow.

Actor(s): System

Dependency / Source: Onboarding

Acceptance Criteria (minimal): Create Family (family name + 4-letter family code)

Suggested Sub-tasks: Create Family (name + 4-letter family code)

Spec Link: matching specification entry

User story example shown as figure_6_1_user_story_example

E2-US01 - Create family

User Story: As a System, I want to create family, so that I can complete the Onboarding workflow.
Description: This story defines the expected behavior for `Create family` in the current product flow.

Actor(s): System

Dependency / Source: Onboarding

Acceptance Criteria (minimal):

- [ ] Create Family (family name + 4-letter family code)

Suggested Sub-tasks:

- Create Family (name + 4-letter family code)

Spec: Specs for E2-US01

Figure 6-1. Example of the FamilyJoy user story record structure

In this project, the user story screenshot functions as planning evidence because it shows that each story was recorded in a structured and traceable format, including a clear identifier, epic association, role and dependency context, minimum acceptance criteria, and a link to the matching specification.

6.3 Sprint and Timeline Planning

In this project, sprint planning was mainly used to advance development in stages. The planning files and sprint CSV records show how different features were grouped, scheduled, and then adjusted according to actual progress. They do not only record what was done when; they also show how the project kept its rhythm and direction. When making sprint-priority decisions, I paid more attention to whether the theme function of each stage was complete, and I preferred validating the overall project workflow through necessary system functions rather than pursuing too many expanded features within a single stage.

The epic-level timeline explains how the final planning direction diverged from the initial proposal while still preserving the core product goal. Appendix B provides the supporting planning evidence behind that figure.

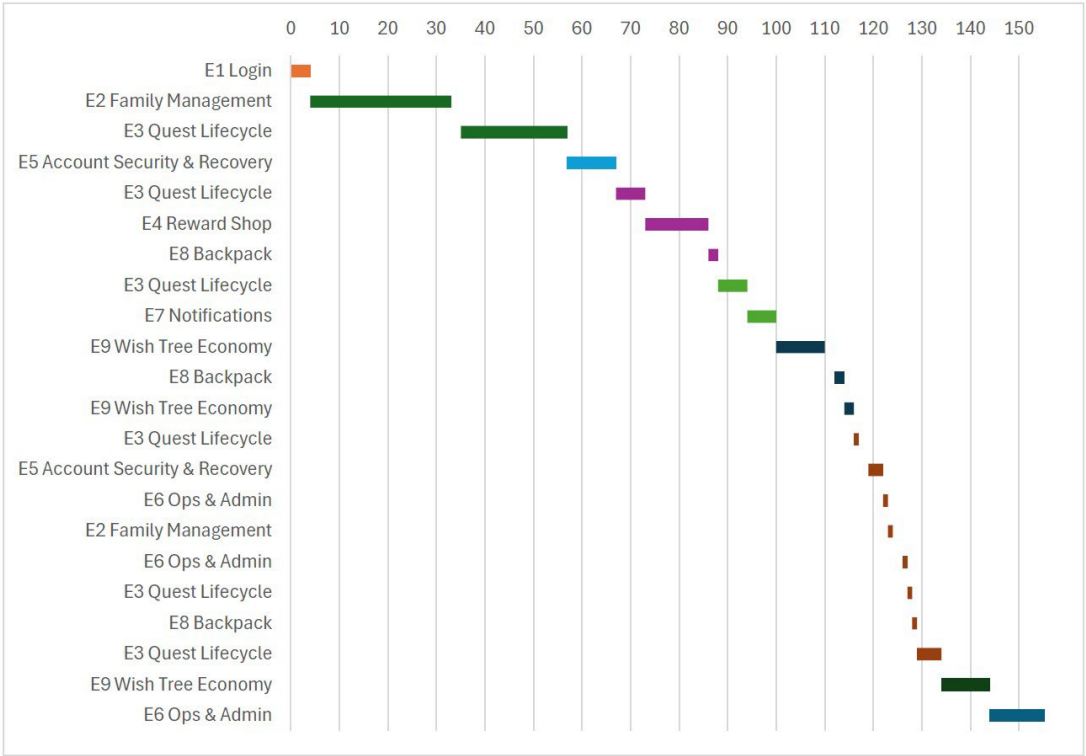


Figure 6-2. FamilyJoy epic-level timeline planning view

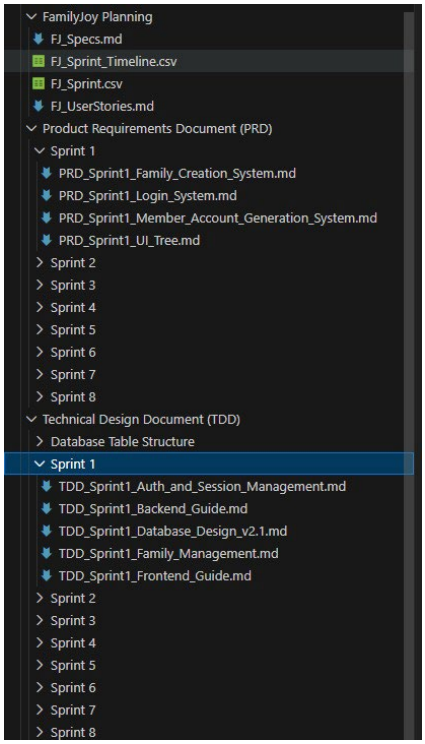


Figure 6-3. Planning document structure evidence

This figure shows how planning records were organised across user stories, sprint files, PRD, and TDD materials, and why the planning evidence is treated as part of the project delivery record rather than as a separate administrative archive.

7. SDLC and Project Delivery Process

The whole project followed an iterative process. Planning, implementation, checking, and documentation reappeared across multiple sprints, while the feature scope gradually moved from the most basic family task loop toward a broader supporting set of modules.

7.1 Development Methodology

In practice, the project is closer to a combination of lightweight iterative development and documentation-driven control. It is not a strict enterprise-level SDLC, but the process is still systematic. Requirements were fixed through PRDs, technical structure was explained through TDD documents, and testing and delivery were recorded through planning materials and BTD summaries.

7.2 Sprint-Based Planning and Execution

Sprints gave the project a stable rhythm. Features were first grouped by stage, then implemented progressively, and then checked against intended user flows. This allowed development to remain aligned with product goals instead of losing direction during implementation.

The following figure records the Scrum rhythm and task organisation used during the planning stage of the project:

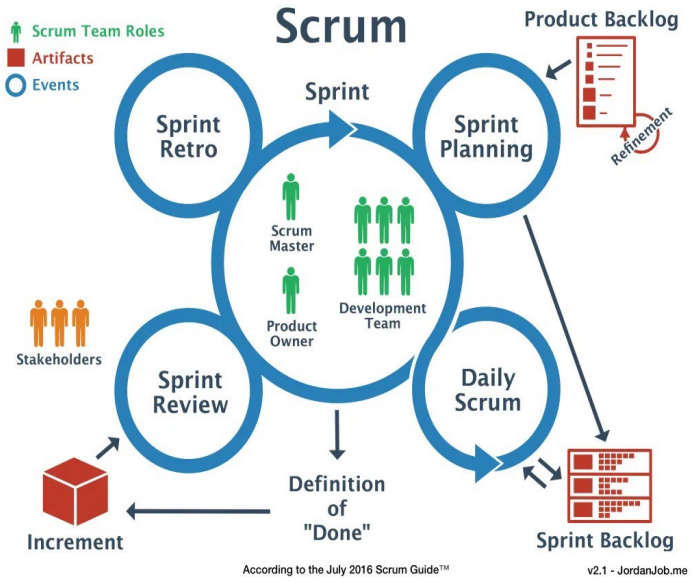


Figure 7-1. Scrum planning evidence

Image source: Figure 1. (2019). Image from "How Scrum helped turn around our development process". Retrieved from https://medium.com/@jw207427/how-scrum-help-turn-around-our-development-process-dac6ff7c700

7.3 Documentation-Driven Development

One of the stronger aspects of the project is that documentation was not added only at the end for submission. PRD documents defined what to build, TDD documents explained how to build it, and BTD records documented sprint-level black-box testing. It was exactly this chain that kept the relationship between scope change, implementation process, and final delivery relatively clear.

7.4 Iterative Refinement from MVP to Expanded Features

The earliest priority was to make the family task loop itself work correctly. Only after that core flow stabilised did the project continue to add reward circulation, shop interaction, backpack handling, wishes, spirit tree support, calendar support, mailbox visibility, and admin governance. This style of growth was controlled expansion rather than uncontrolled feature stacking.

7.5 Individual Role Coverage and Transferable Skills

Although this is an individual project, the work involved is not narrow. Requirements analysis, sprint planning, interface and workflow decisions, back-end and front-end implementation, testing, technical writing, and delivery preparation were all handled by the same person. As a result, the project demonstrates transferable skills more convincingly through multi-role responsibility within an individual project than through team-based contribution language.

From the perspective of actual effort, the most important part was not coding itself, but the early design stage. Compared with writing code, early design directly affected the stability of later development. The most time-consuming part was system-design output, because there were many possibilities and many changes, and a large amount of time had to be spent reasoning through them. The most difficult part was also the technical-design stage, because it had to consider how different technical options would fit the system design while also balancing stability, complexity, and future extensibility. Across development, documentation, and testing, the balance I most often had to make was to clarify structure and boundaries first, and only then push implementation and validation forward.

7.6 Final Delivery Packaging

The final delivery was never only a code repository. The final submission package also includes PRD, TDD, BTD, user stories, planning artefacts, diagrams, screenshots, Git history evidence, and presentation materials. Delivery packaging itself is part of the project result, not an explanation added at the last minute.

8. Project Timeline

The project timeline in this section serves as the overall delivery view rather than a duplicate of sprint detail. Figure 8-2 shows the broader progression of the project in a form that is easier to read at report level.

This timeline shows how the project moved from proposal-stage expectations into actual implementation stages. The more detailed epic-level planning view is discussed in Section 6 and retained in Appendix B.

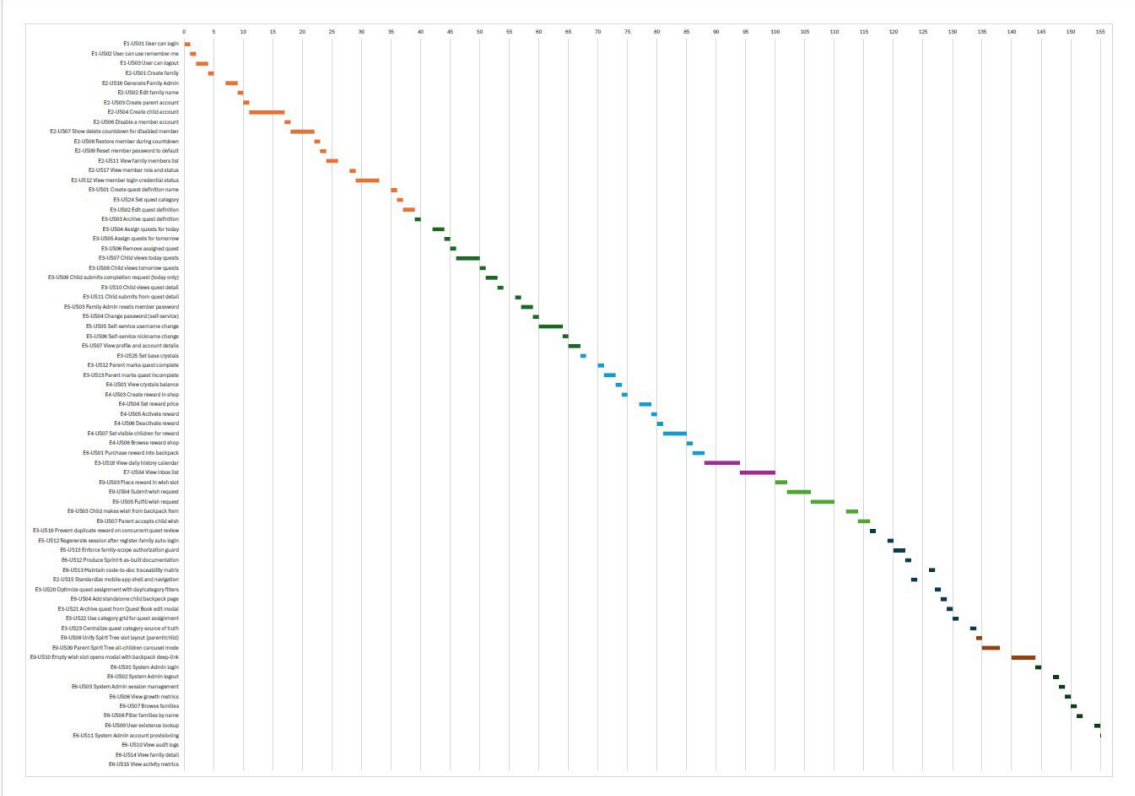


Figure 8-2. FamilyJoy overall timeline reference

This progression matches the MVP-first strategy and the practical reality of an individual project. Where the plan changed, it changed for understandable reasons: workflow clarity, complexity control, time management, and the need to keep related functions integrated instead of scattering effort too widely.

9. Final System Overview

The finished FamilyJoy system can be understood as a role-based household workflow application with one admin extension. Its main logic revolves around a closed product loop:

parents define and assign routine work, children complete and submit it, parents review the results, and the system updates reward-related state that affects later child interaction.

9.1 Role Overview

Parent users manage the structure of family activity. They create members, define quests, assign work, review submissions, and manage reward settings. Child users operate within a simpler, action-oriented interface. They log in, complete assigned work, submit it, receive crystal outcomes, interact with the shop, view backpack items, create wishes, and review related history. Admin users work in a separate operational area and focus on the dashboard, users, families, and logs.

9.2 Core Functional Modules

The major functional modules are authentication, family management, dashboard routing, quest definition and assignment, child submission, parent review, crystal and reward handling, shop interaction, backpack and wish handling, spirit tree interaction, calendar history, mailbox visibility, and admin governance.

9.3 Closed-Loop Product Logic

The main reason the system feels coherent is that these modules are not isolated. A parent creates a quest template, assigns a dated quest, the child submits completion, the parent makes a decision, and the reward system updates the next stage of child interaction. That closed loop gives the application more structure than a typical family checklist.

9.4 Role-Based Showcase Mapping

For reporting and explanation purposes, the product can be described through four visible states: onboarding or new-account state, parent workflow state, child workflow state, and admin governance state. That mapping helps readers understand the system from actual usage rather than only from architecture diagrams.

10. Design Documents

The design focus of FamilyJoy is not to turn the system into a generic CRUD application. The important question is how to keep a multi-role, multi-state family process clear, stable, and maintainable. The key design decisions mainly appear in the layered runtime structure, the relational database design, the state-based business logic, and the explicit separation between the family-facing side and the admin side.

10.1 Architecture Design

The system uses Node.js and Express on the server side, and EJS in the rendering layer. In actual runtime terms, it broadly follows the chain route -> controller -> service -> repository -> database -> viewModel -> EJS. This layered structure is well suited to the current project because it avoids mixing HTTP handling, business logic, data access, and view preparation into one place.

The admin module was also separated structurally instead of simply inserting admin as another role into the same entry path. That decision is important because it keeps system-governance logic and household-interaction logic clearly separated, which is better both for security and for later explanation. The admin side was not separated just to create another interface; it was separated in order to achieve boundary isolation in both access and governance authority.

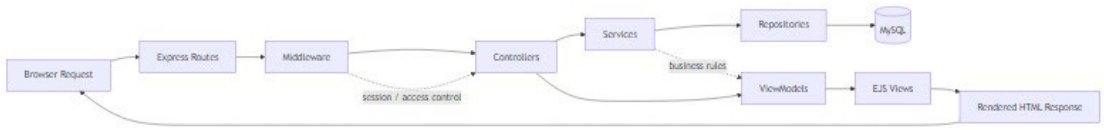


Figure 10-1. SSR runtime flow

10.2 Database Design

The FamilyJoy database is organised mainly around several related domains: families and users, sessions and login events, quest definition and daily quest records, rewards and crystal movement, backpack items, wishes, spirit tree data, mailbox records, and admin information. According to the documented database structure, the family boundary is the top ownership boundary for most business data.

For a system of this kind, family as the top-level boundary is necessary, because the product fundamentally operates as a household system. Most business data should belong to the family first, and only then be distributed within that family to parent and child accounts. Only after family is established as the top access boundary do later access control, data isolation, and page behaviour remain clear. Compared with a looser document-style structure, a relational schema is easier to reason about, easier to validate, and easier to maintain.

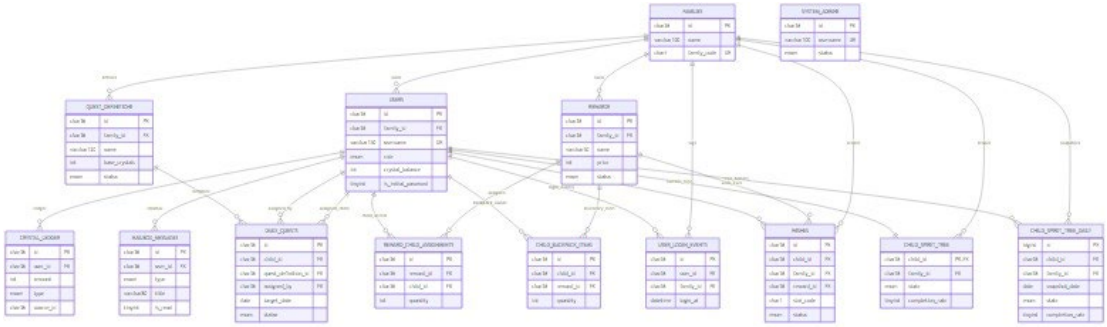


Figure 10-2. FamilyJoy database ER diagram

10.3 Data Structures and Core Logic Techniques

The project makes extensive use of relational data structures. Families, users, quests, rewards, wishes, mailbox entries, and logs are all modelled as entities with explicit relationships rather than as unstructured data. This directly supports access control, state consistency, and later summarisation.

The core logic is mainly built around state transitions. Quests move through assignment, submission, and review stages. Crystal and reward updates use a ledger-style approach instead of simply overwriting final values. Role-based access checks depend on both the user role and the family boundary. Calendar and history pages use aggregation to turn raw records into readable information. These design choices show that the project did not merely render pages, but implemented a relatively complete set of data structures and business logic. Reward and crystal values were not handled as direct overwrite values because a ledger-like approach is more suitable for future system extension and also better aligned with possible later use in data analysis.

Technique / Pattern	Where Used	Why It Fits
Relational entity structure	families, users, quests, rewards, wishes, logs	Keeps ownership clear and preserves data consistency
State-based workflow logic	quest lifecycle and review process	Makes state change explicit and testable
Ledger-style tracking	crystal and reward updates	Preserves traceability of value updates
Role and boundary checks	user and admin access control	Protects record scope and route boundaries
Aggregation logic	calendar and history views	Turns raw records into readable summaries
State-update flow	wish and spirit tree behaviour	Connects user actions to visible results

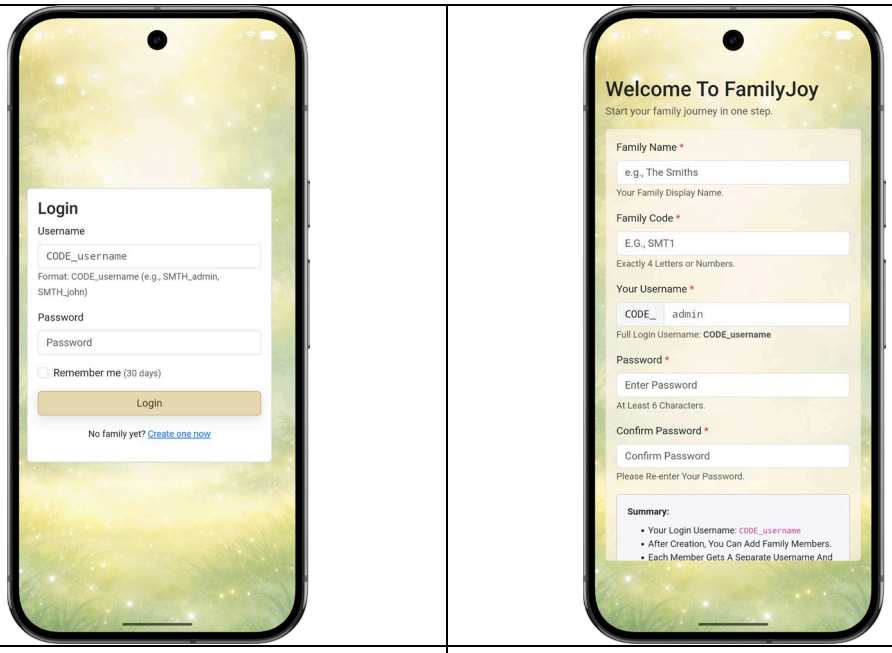
10.4 Interface and UI Design

The system interface is organised by role and functional groups rather than as a flat navigation set. Parent pages emphasise management and decision-making, child pages emphasise action, feedback, and visible progress, and admin pages remain separate. Because of this, the system feels more like a product with a clear usage scenario than a random collection of pages.

This section is supported by a small set of representative user-facing screenshots rather than many similar screens. Together they show onboarding, parent-side management, child-facing task interaction, and child-facing result visibility without overloading the page with repeated views.

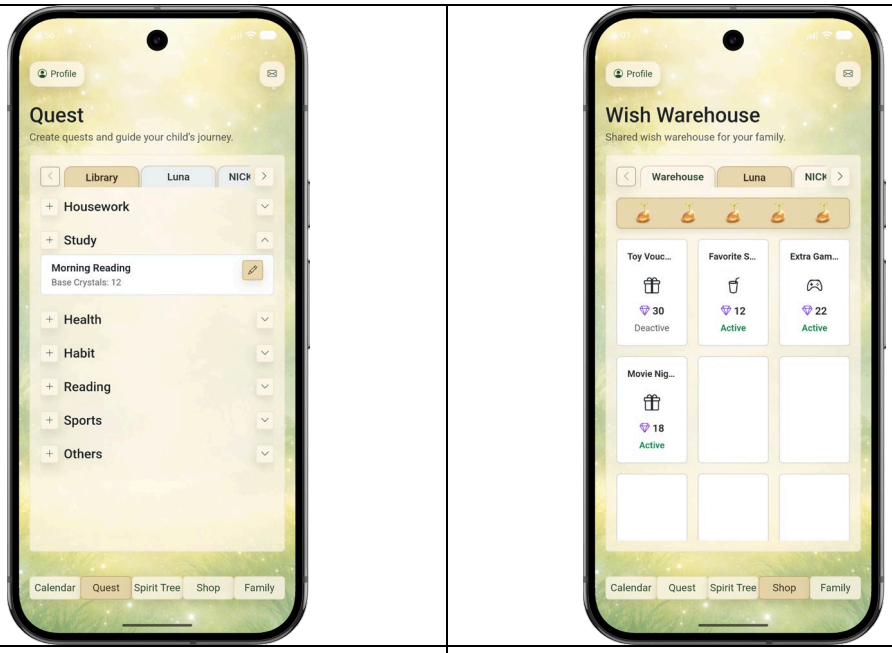
The screenshots below focus on six user-facing interfaces chosen for variety: login, parent registration, parent quest management, parent reward management, child quest access, and

child backpack visibility. Two admin-facing screenshots are kept separate so that governance views are not visually mixed with the family-facing user flow.



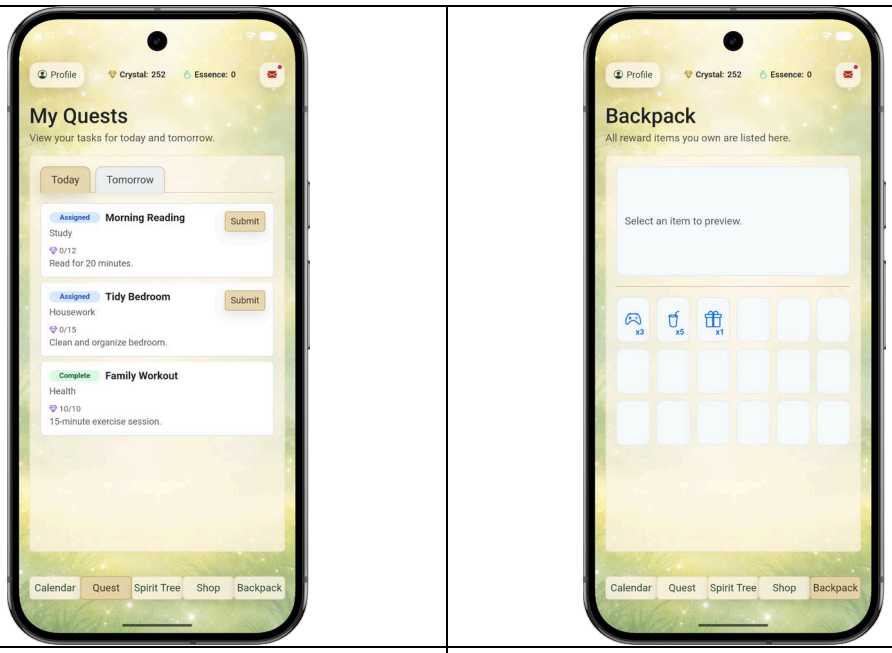
Login page	Parent registration

Figure 10-3. User interface set A: login and parent registration



Parent quest library	Parent reward management

Figure 10-4. User interface set B: parent quest and reward management



Child quest view	Child backpack view

Figure 10-5. User interface set C: child quest and backpack views

For the admin-facing side, the dashboard and users page together are enough to show the separate governance area without mixing it with the family-user screens.

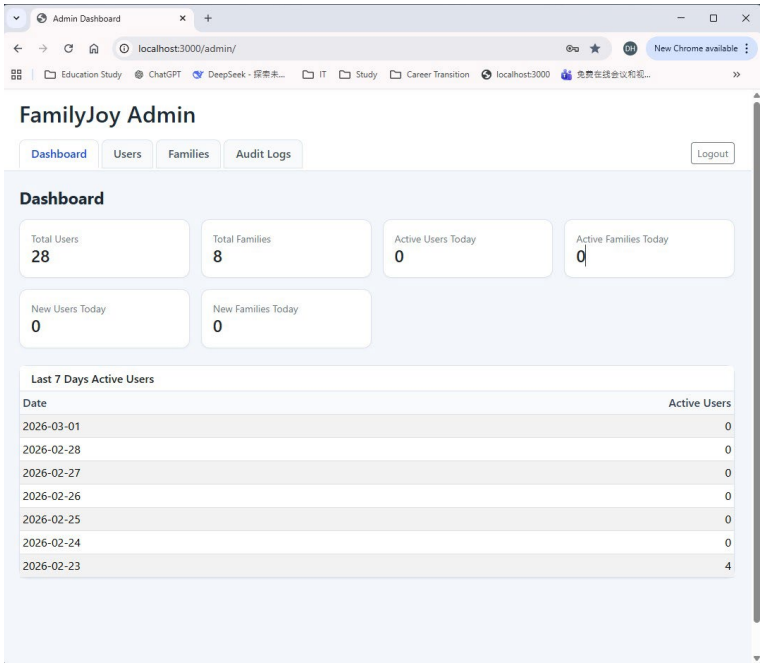
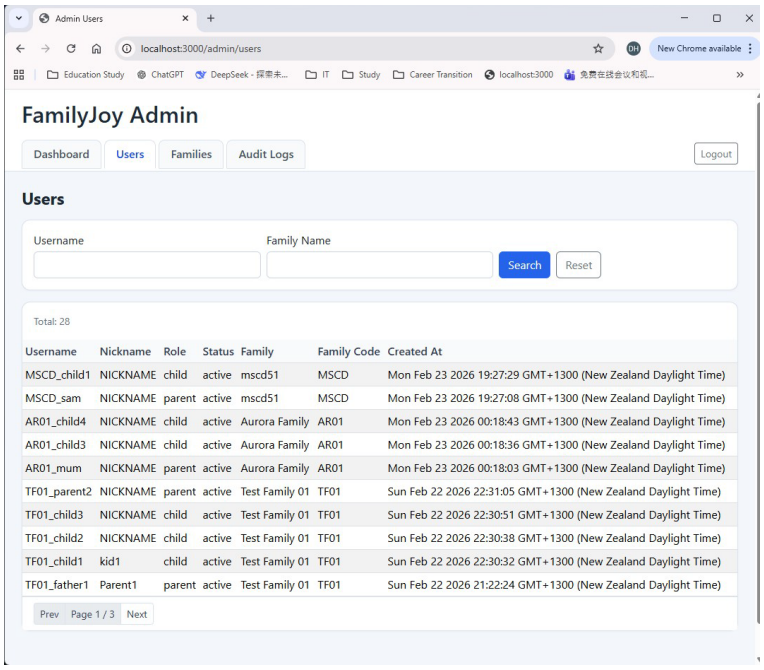


Figure 10-6. Admin dashboard page



Username	Nickname	Role	Status	Family	Family Code	Created At
MSCD_child1	NICKNAME	child	active	mscd51	MSCD	Mon Feb 23 2026 19:27:29 GMT+1300 (New Zealand Daylight Time)
MSCD_sam	NICKNAME	parent	active	mscd51	MSCD	Mon Feb 23 2026 19:27:08 GMT+1300 (New Zealand Daylight Time)
AR01_child4	NICKNAME	child	active	Aurora Family	AR01	Mon Feb 23 2026 00:18:43 GMT+1300 (New Zealand Daylight Time)
AR01_child3	NICKNAME	child	active	Aurora Family	AR01	Mon Feb 23 2026 00:18:36 GMT+1300 (New Zealand Daylight Time)
AR01_mum	NICKNAME	parent	active	Aurora Family	AR01	Mon Feb 23 2026 00:18:03 GMT+1300 (New Zealand Daylight Time)
TF01_parent2	NICKNAME	parent	active	Test Family 01	TF01	Sun Feb 22 2026 22:31:05 GMT+1300 (New Zealand Daylight Time)
TF01_child3	NICKNAME	child	active	Test Family 01	TF01	Sun Feb 22 2026 22:30:51 GMT+1300 (New Zealand Daylight Time)
TF01_child2	NICKNAME	child	active	Test Family 01	TF01	Sun Feb 22 2026 22:30:38 GMT+1300 (New Zealand Daylight Time)
TF01_child1	kid1	child	active	Test Family 01	TF01	Sun Feb 22 2026 22:30:32 GMT+1300 (New Zealand Daylight Time)
TF01_father1	Parent1	parent	active	Test Family 01	TF01	Sun Feb 22 2026 21:22:24 GMT+1300 (New Zealand Daylight Time)

Figure 10-7. Admin users page

10.5 Key Workflow Diagrams

The workflow diagrams most worth keeping in the report are usually the ones that can explain repeated behaviour clearly, such as registration and first login, the quest lifecycle, the shop and reward flow, the backpack-to-wish flow, the spirit tree review flow, the calendar support flow, and admin runtime interaction. Compared with plain text description, these diagrams are better suited to showing how state changes step by step.

The existing workflow diagram set already covers the main behaviour families, including registration and login, quest flow, shop flow, spirit tree and backpack interaction, calendar support, and code-structure overview. The main task is therefore to place those diagrams in the right report sections and use supporting workflow screenshots where a lighter visual reference is sufficient.

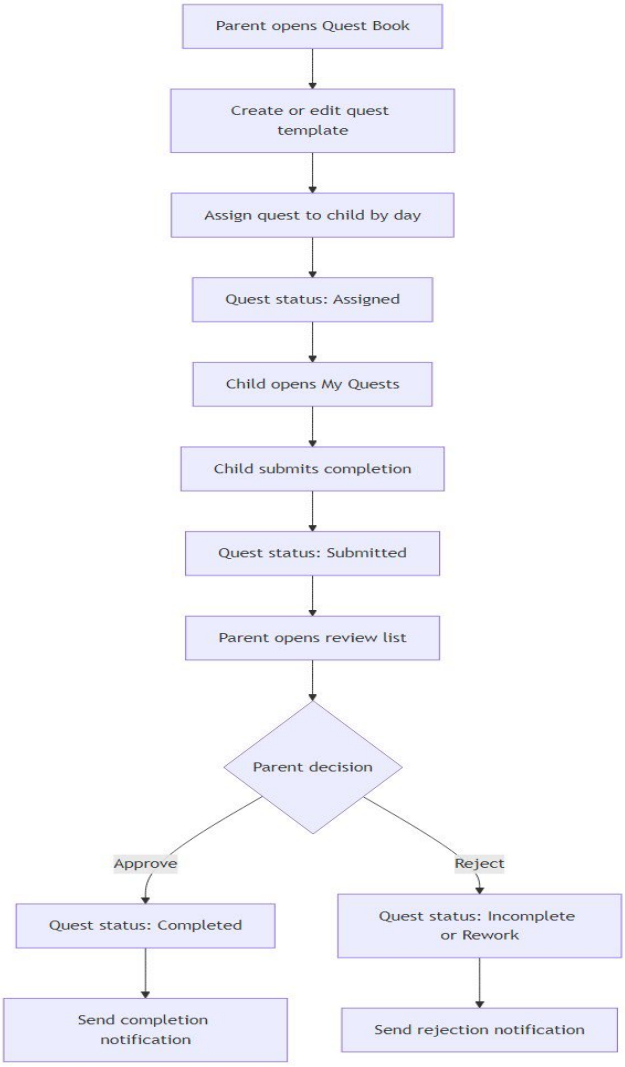


Figure 10-8. Quest workflow

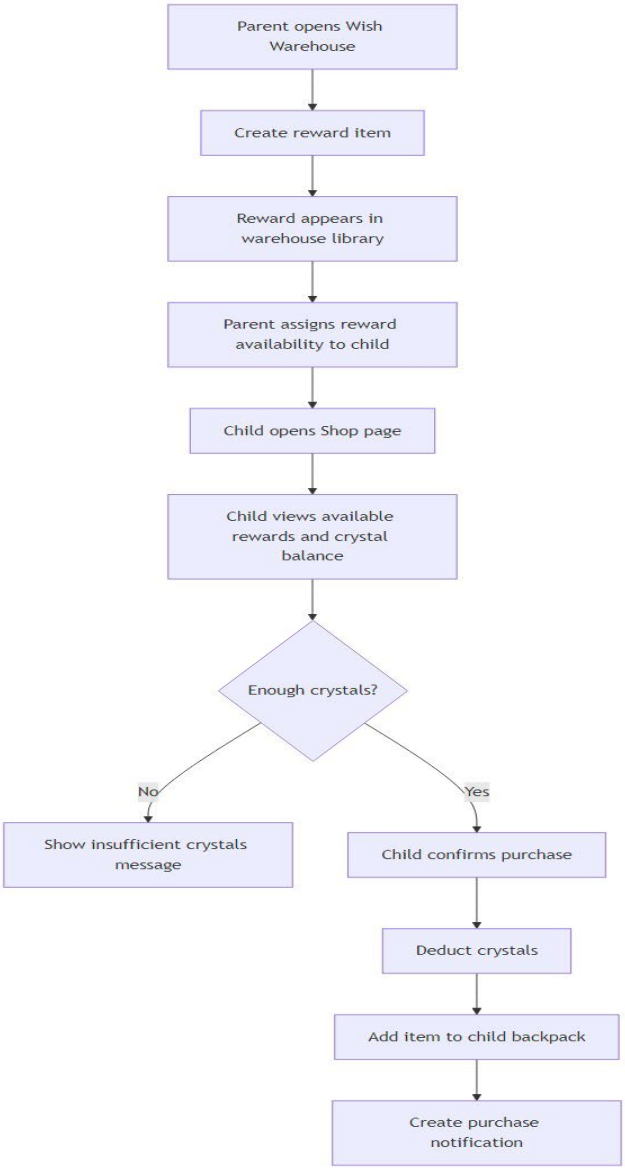


Figure 10-9. Shop workflow

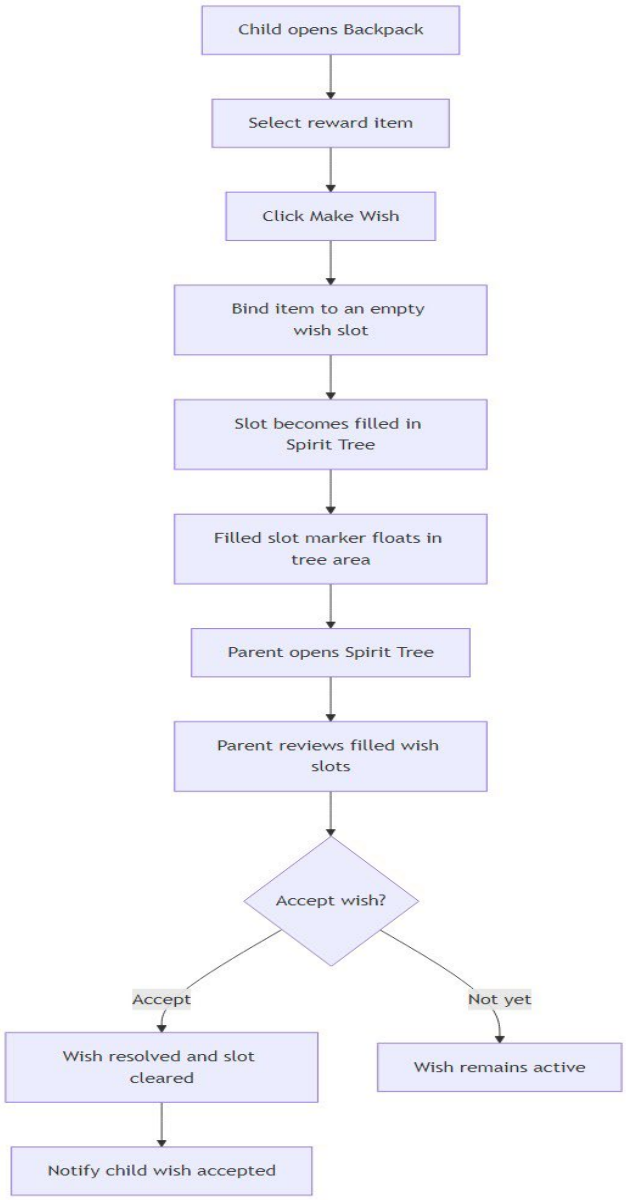


Figure 10-10. Spirit Tree workflow

10.6 Security and Access Design

System security is mainly implemented through session-based authentication, route protection, family-bound access logic, forced password change at first login, and a separate admin entry path. None of these mechanisms are individually complex, but when combined they create a relatively clear access boundary. For a project of this scale, that is already an appropriate level of design.

10.7 Non-Functional Design

From a non-functional perspective, the system mainly focuses on maintainability, usability, extensibility, and safe defaults. The layered structure improves maintainability, the role-based page organisation improves usability, session handling and route protection provide safer default behaviour, and modular services together with a clear schema preserve room for future expansion.

11. Developer Documentation

From a developer perspective, this codebase already needs clear explanation, but it is not yet so large that it cannot be understood at a high level. The overall structure can be divided into three main areas: the server, the family-facing client layer, and the admin module.

11.1 Directory Structure

The top-level development structure can be summarised as follows:

Path	Main Responsibility
workspace/src/familyjoy_server	Main server runtime, routes, controllers, services, repositories, middleware, config
workspace/src/familyjoy_client	EJS views, front-end scripts, layouts, and role-facing pages
workspace/src/familyjoy_admin	admin routes, controllers, views, and governance-related logic

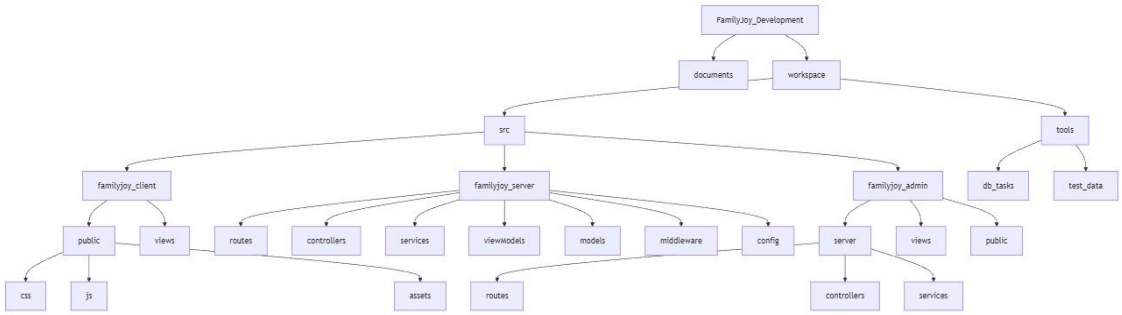


Figure 11-1. Route and module responsibility map

11.2 Server Layer Responsibilities

The server layer handles request entry, workflow coordination, business rules, persistence, and view-data preparation. Routes expose feature entry points, controllers organise request and response handling, services carry the main business logic, repositories handle database access,

viewModels prepare SSR rendering data, and middleware is responsible for access control and error handling.

11.3 Client Layer Responsibilities

The client layer is rendered through EJS and enhanced where necessary with lightweight JavaScript. Because the project remains SSR-based, browser-side scripts mainly support interaction rather than acting as a full SPA runtime. This choice fits the project scope and keeps front-end complexity under control.

11.4 Admin Layer Responsibilities

The admin side is intentionally separated. Its job is not to participate in the family task workflow, but to provide system-level visibility. It mainly covers dashboard display, user inspection, family inspection, and log review.

11.5 Key Modules to Explain in Detail

The most important implementation modules include authService.js, familyService.js, questService.js, shopService.js, wishService.js, spiritTreeService.js, calendarService.js, mailboxService.js, and accessGuard.js. Together they cover authentication, boundary control, the core workflow, child-facing interaction, and route protection.

11.6 Development Workflow

From the perspective of code evolution, the project was pushed forward sprint by sprint while remaining aligned with PRD and TDD materials. This development rhythm is visible in the repository structure, module organisation, and the way features were added, and it prevented the system from collapsing into a hard-to-maintain monolithic file.

12. Tools and Technologies Used

The technology stack used in the project remained within the approved boundary and matched the product goals. Node.js and Express were used to support server-side routing and workflow control, EJS was used to keep rendering inside an SSR model, MySQL provided relational persistence for the family-bound data model, and Bootstrap supported practical interface construction without requiring a heavier front-end framework.

The choice of this stack was deliberate. A full SPA architecture would have increased front-end state management and delivery complexity, while the selected SSR approach made it easier to keep authentication, view rendering, and business workflow in one maintainable structure. This was especially important in an individual project where the main constraint was not theoretical capability, but the need to keep implementation and explanation aligned.

The project also depended on planning and delivery tools rather than code tools alone. User stories, sprint CSV files, PRD, TDD, and BTD documents acted as project-management artefacts, while Git and GitHub evidence showed that version control was genuinely used throughout development. In that sense, the tools section covers both runtime technology and the workflow technology that supported delivery discipline.

Technology	Role in Project	Reason for Use
Node.js	Runtime	Provides the server-side JavaScript runtime
Express.js	Web framework	Provides a clear route and middleware structure
EJS	Templating	Supports SSR pages without introducing SPA complexity
Bootstrap	UI styling	Improves efficiency of layout and components
MySQL	Database	Fits the project's relational ownership and workflow model
express-session / express-mysql-session	Session management	Supports persistent SSR authentication and controlled sessions
Vanilla JavaScript	Browser interaction logic	Keeps front-end behaviour lightweight
Capacitor SDK	Hybrid support	Preserves a possible Android packaging path
Git / GitHub	Version control	Preserves development history and workflow evidence
Markdown / Mermaid	Documentation and diagrams	Supports technical documentation and visual explanation

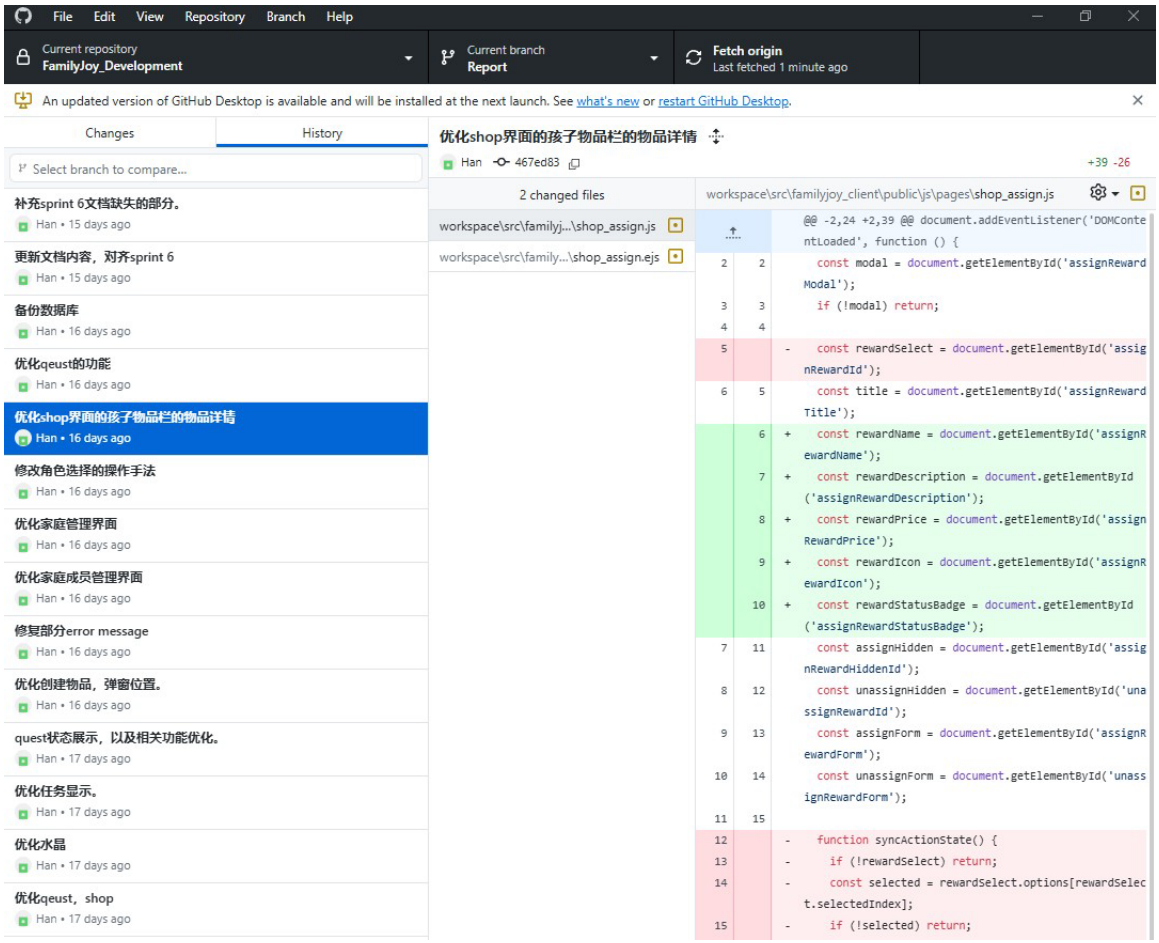


Figure 12-1. Git commit history evidence

13. Deliverables and Software Overview

This section distinguishes between the delivered software system and the wider assessment portfolio submitted alongside it, because the final submission is evaluated as a complete project package rather than as code alone.

From the software side, the delivered system includes the family-facing workflow and the separate admin governance module. The family-facing side covers authentication, family management, quest definition and assignment, child submission, parent review, reward and crystal handling, shop interaction, backpack and wish handling, Spirit Tree interaction, calendar history, mailbox visibility, and related profile pages. The admin side remains separate and provides dashboard, users, families, and logs views. Taken together, the delivered system is clearly beyond a minimal prototype and represents a complete role-based workflow application.

From the portfolio side, the submitted materials include PRD documents, TDD documents, BTD sprint summaries, user stories, sprint planning records, database-structure documentation,

diagrams, screenshots, version-control evidence, and presentation materials. The final presentation PPT is already part of the submission set. The presentation video clip is also included as a supporting deliverable under the filename DongHan WU - Video Clip - MCSD51 - 20260223.mp4.

These materials do more than complete the file set. They show that the project was planned, implemented, tested, and packaged through a structured process rather than assembled only at the end. The deliverables section therefore summarises how the finished software and the supporting evidence fit together as one final portfolio.

14. Testing and Validation

The project mainly used manual black-box testing, which is appropriate for the current system. The focus was on confirming whether the role-based workflow behaved as expected in the actual interface rather than splitting internal logic into a large number of code-level tests.

14.1 Testing Strategy

The main focus of testing is user-visible behaviour, including login, navigation, form submission, state changes, access restrictions, and end-to-end workflow results. This strategy matches the project itself, because the most critical questions are exactly what the user can do, what the user can see, and whether the next state appears correctly. For the current project, a black-box approach is more appropriate because it needs to simulate real user behaviour to validate the system rather than first investing heavily in code-level test construction.

Sprint-level BTD documents are used as lightweight evidence. The current project scale does not require an overly complex testing method, and these summaries are sufficient to support validation of the main functions.

14.2 Core Test Areas

The main test areas include family registration, member creation, login and logout, first-password change, quest template creation, quest assignment, child submission, parent review, reward purchase, backpack visibility, wish creation, spirit tree state updates, calendar history rendering, mailbox visibility, and admin access behaviour.

Feature Area	Test Type	Result	Note
Authentication and first login	Manual black-box	Completed	OK
Family and member management	Manual black-box	Completed	OK
Quest creation and assignment	Manual black-box	Completed	OK

Child submission and parent review	Manual black-box	Completed	OK
Reward, shop, and crystal behaviour	Manual black-box	Completed	OK
Backpack, wish, and spirit tree flow	Manual black-box	Completed	OK
Calendar and mailbox views	Manual black-box	Completed	OK
Admin access and governance views	Manual black-box	Completed	OK

14.3 Test Results

The sprint-level BTD records show that the core delivered functions were manually validated through the visible interface and checked against expected behaviour. Across the delivered scope, the main workflow areas used in the final system were verified at least once during sprint-level testing, and no unresolved failures remained in the final submission baseline. For the current project scale, this provides a sufficient validation basis when read together with the UI screenshots, workflow figures, and implementation traceability described elsewhere in the report.

The validated result areas include authentication and first-login handling; family and member setup; quest creation, assignment, submission, and review; reward, shop, backpack, and wish behaviour; spirit tree and feedback updates; calendar and mailbox visibility; and separate admin access behaviour. These result areas match the main delivered modules rather than isolated technical fragments.

In addition to the BTD summaries, the existing workflow images and diagrams also function as supporting validation evidence. Quest flow, shop flow, spirit tree flow, and calendar flow materials show that the main user-visible workflow was checked in practice. They do not replace the testing records, but they strengthen the visual side of the validation evidence.

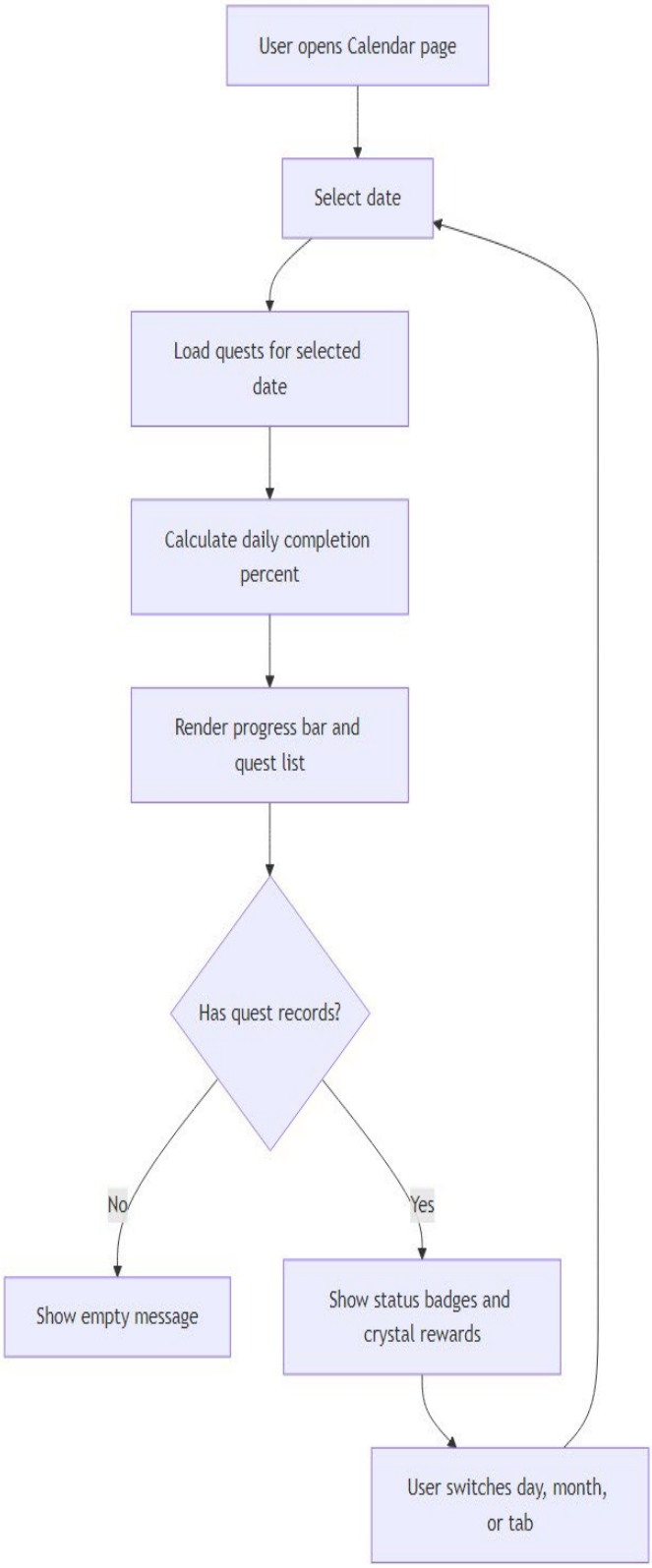


Table 14-1. Core validation summary

Validated Area	Evidence Status
Authentication and first login	Validated through sprint-level BTD summary and UI verification
Family and member setup	Validated through sprint-level BTD summary and UI verification
Quest assignment and review flow	Validated through workflow checks and BTD coverage
Reward, shop, backpack, and wish flow	Validated through sprint-level BTD summary and visible state checks
Spirit Tree, calendar, and mailbox visibility	Validated through UI checks and supporting workflow evidence
Admin access and governance pages	Validated through admin access checks and page-level review

Figure 14-1. Calendar flow as supporting validation evidence

FamilyJoy Shop Flow (Parent Warehouse and Child Shop)

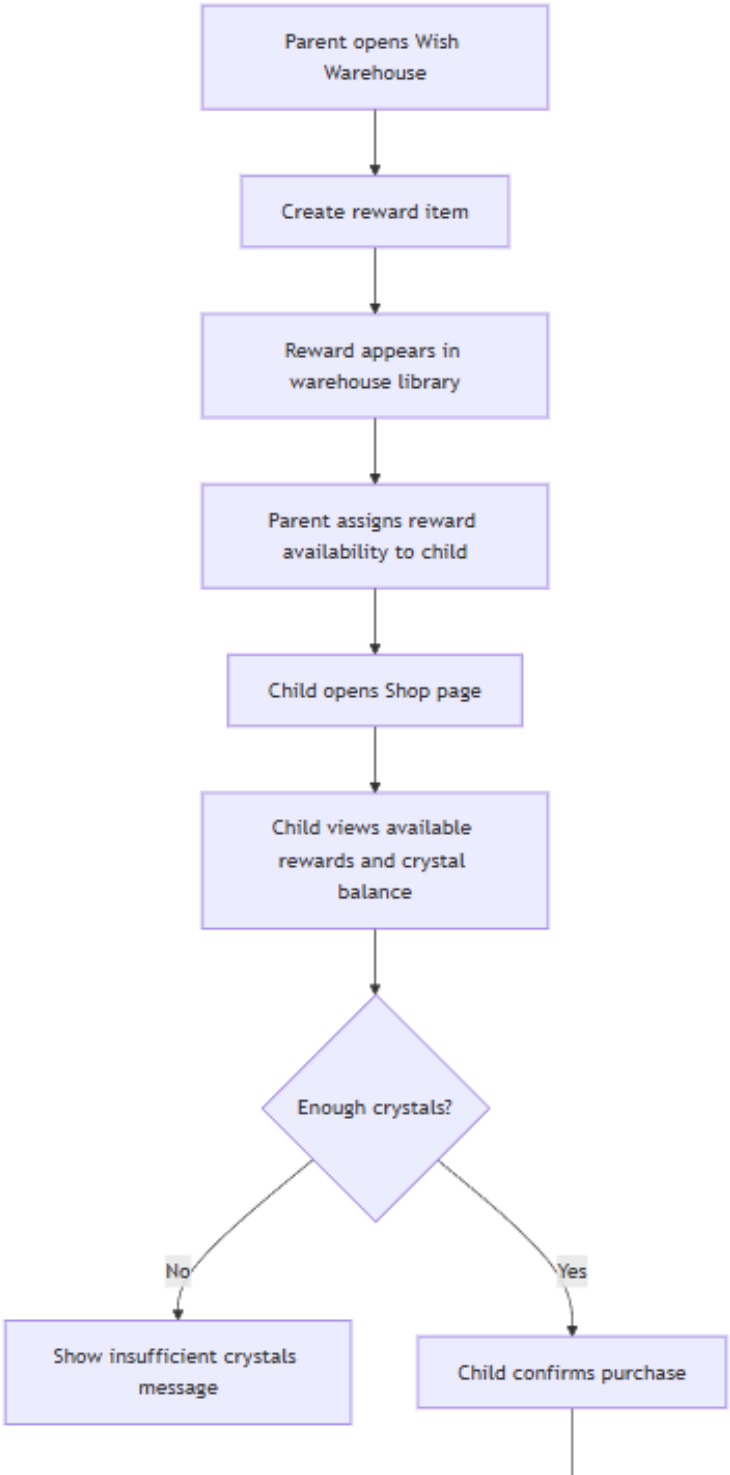


Figure 14-2. Shop flow as supporting validation evidence

15. User Guide

This section explains the system from the perspective of actual use rather than implementation.

15.1 Parent Guide

Parent users first register a family or log in to an existing family account. After entering the system, they can create family members, define quest templates, assign tasks by date, review child submissions, manage rewards, and view related pages such as wishes, spirit tree, calendar, and mailbox. The parent side therefore acts as the management and decision-making side, and it drives the whole workflow.

Representative parent-facing screenshots include onboarding, family management, quest management, child-task review, reward management, wish and reward storage, spirit tree view, calendar history, and parent profile pages.

15.2 Child Guide

Children log in using a member account, and if it is their first login they must first change the initial password. After that, they can view assigned quests, submit completion results, receive crystal updates, enter the shop, view backpack items, create wishes, and review history through calendar and mailbox-related pages. The child-facing interface is intentionally kept simpler and more action-oriented.

Representative child-facing screenshots include quest view, shop interaction, backpack view, spirit tree view, calendar history, and mailbox pages.

15.3 Admin Guide

Admins enter through a separate admin path rather than through the normal family-user login route. After login, they can access the dashboard, users page, families page, and logs page. These functions are intended for governance and visibility rather than for household task interaction.

Representative admin-facing screenshots include dashboard, users, families, and audit-log views.

16. Configuration and Deployment Guide

The project uses the workspace structure for local runtime configuration. The environment requires Node.js, npm, and MySQL, and the database must be initialised before the application can run correctly. Exact commands are documented in the README, while the deployment logic itself is straightforward: configure the environment, initialise the schema, install dependencies, and start the server process.

The key environment variables include SESSION_SECRET, PORT, DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, and where required DB_PORT. After the schema has been applied and the dependencies have been installed, the main application can be accessed through the local runtime address, while the admin portal is entered through the /admin path. This separate entry path is part of the intended access design rather than an accidental routing choice.

This deployment approach matches the scale of the project. It is designed for local setup and assessment verification, which aligns with the delivered scope of the system and the way it was developed and tested.

17. Future Enhancements

There are several natural enhancement directions beyond the current delivered scope. Reusable templates and recurring quests could reduce the repeated cost to parents of configuring tasks. Richer child submission forms, such as image uploads or notes, could improve review quality. Wish handling and Spirit Tree progression could also become more continuous and interactive, while analytics and trend reporting would help parents observe long-term habit development.

More detailed data-analysis presentation already appeared in the planning stage, but it was deliberately postponed because of the trade-off between output value, available time, implementation complexity, and maintenance cost. The same applies to reminders, more detailed admin reporting, and more complete mobile-friendly behaviour. These directions were not ignored; they were intentionally placed into a later stage so that the delivered version could remain internally consistent and complete at its current scale.

18. Conclusion

The starting point of FamilyJoy was simple, but the problem was real: when the assignment, completion, and reward fulfilment of family tasks rely for a long time on temporary verbal communication, the whole process becomes difficult to sustain in a fair and stable way. The final

project turns those stages into a connected, state-explicit workflow through a role-based system.

I believe the most valuable part of this project is not that any one page is especially complex, but that under controlled development complexity it completed the necessary system functions and the entire main workflow. The process truly connects family setup, quest assignment, child submission, parent review, reward circulation, and later interaction features. At the same time, planning records, technical documents, BTD testing summaries, and version-control evidence all support the final software result.

From a design-judgement perspective, the Spirit Tree display proved to be the right decision. Although the quest system is the main system, its visual feedback by itself is not strong enough. It needs a growth-oriented display such as Spirit Tree to strengthen the user's intuitive sense of the result. If I were to redesign it in the future, I might try another more personalised presentation style, for example something closer to a digital-pet visual style.

The project can continue to expand, and the final submission quality still depends on whether appendix materials are organised clearly. Even so, the current system provides a coherent project outcome that addresses the original problem, demonstrates relatively complete technical implementation ability, and shows that an individual project can take on multiple development roles when the process is managed clearly enough.

References

External Sources

- Node.js. Official documentation. https://nodejs.org/docs/latest/api/
- Express.js. Official documentation. https://expressjs.com/
- EJS. Official documentation. https://ejs.co/
- MySQL. Official documentation. https://dev.mysql.com/doc/
- Bootstrap. Official documentation. https://getbootstrap.com/docs/
- Figure 1. (2019). Image from "How Scrum helped turn around our development process". Retrieved from https://medium.com/@jw207427/how-scrum-help-turn-around-our-development-process-dac6ff7c700

Internal Project Documents

- FamilyJoy Planning directory, including user stories, specs, sprint records, and sprint timeline files.
- Product Requirements Document (PRD) directory, including sprint-specific requirement files.

- Technical Design Document (TDD) directory, including database structure documentation and related design assets.
- Black-box Test Document (BTD) directory, including sprint-level validation summaries.

Appendix

The appendix groups the main supporting evidence used throughout the report and explains how each evidence set relates to planning, implementation, testing, and submission. It does not attempt to reproduce every source file in full. Instead, it presents the most relevant evidence groups in a controlled and traceable way so that the claims made in the main body can be checked against retained project records.

Appendix A. Initial Proposal

- Deliverable Document (DVL)/Resource/MCSD51_Project Proposal_DongHan WU.pdf

Notes:

- This file is retained as the original submitted project proposal.
- The proposal summary and proposal-change sections in the report are based on this document rather than reproducing it page by page.

Appendix B. Project Planning Artifacts

- FamilyJoy Planning/FJ_UserStories.md
- FamilyJoy Planning/FJ_Specs.md
- FamilyJoy Planning/FJ_Sprint.csv
- FamilyJoy Planning/FJ_Sprint_Timeline.csv
- Deliverable Document (DVL)/Figures/figure_6_2_timeline_epic.jpg
- Deliverable Document (DVL)/Figures/figure_8_2_timeline_sprint.jpg

Notes:

- These planning files form the evidence base for the project-planning discussion in the main body. Together they show how epics, user stories, sprint scheduling, and the timeline evolved after the original proposal and how the final development direction was organised in practice.
- FJ_Sprint.csv mainly reflects staged task organisation.
- Figure 6-2 supports the changed epic-level planning view discussed in Section 6.
- FJ_Sprint_Timeline.csv supports the planning progression described together with the timeline figure in the main report.

- Figure 8-2 is the preferred overall timeline reference for Section 8, while Figure 6-2 remains the more detailed planning evidence.

Appendix C. PRD Evidence by Sprint

- Product Requirements Document (PRD)/Sprint 1/PRD_Sprint1_Family_Creation_System.md
- Product Requirements Document (PRD)/Sprint 1/PRD_Sprint1_Login_System.md
- Product Requirements Document (PRD)/Sprint 1/PRD_Sprint1_Member_Account_Generation_System.md
- Product Requirements Document (PRD)/Sprint 1/PRD_Sprint1_UI_Tree.md
- Product Requirements Document (PRD)/Sprint 2/PRD_Sprint2_Daily_Quest_Assignment.md
- Product Requirements Document (PRD)/Sprint 2/PRD_Sprint2_Family_Member_Modify.md
- Product Requirements Document (PRD)/Sprint 2/PRD_Sprint2_Quest_Book_System.md
- Product Requirements Document (PRD)/Sprint 2/PRD_Sprint2_Quest_Completion.md
- Product Requirements Document (PRD)/Sprint 2/PRD_Sprint2_Quest_Review.md
- Product Requirements Document (PRD)/Sprint 2/PRD_Sprint2_UI_Tree.md
- Product Requirements Document (PRD)/Sprint 3/PRD_Sprint3_Child_Shop_Backpack.md
- Product Requirements Document (PRD)/Sprint 3/PRD_Sprint3_Crystal_Wallet_Ledger.md
- Product Requirements Document (PRD)/Sprint 3/PRD_Sprint3_Quest_Rewards.md
- Product Requirements Document (PRD)/Sprint 3/PRD_Sprint3_Reward_Shop_Management.md
- Product Requirements Document (PRD)/Sprint 3/PRD_Sprint3_UI_Tree.md
- Product Requirements Document (PRD)/Sprint 4/PRD_Sprint4_Calendar.md
- Product Requirements Document (PRD)/Sprint 4/PRD_Sprint4_Notifications.md
- Product Requirements Document (PRD)/Sprint 4/PRD_Sprint4_Overview.md
- Product Requirements Document (PRD)/Sprint 4/PRD_Sprint4_Reward_Icons.md
- Product Requirements Document (PRD)/Sprint 4/PRD_Sprint4_UI_Tree.md
- Product Requirements Document (PRD)/Sprint 5/PRD_Sprint5_Overview.md
- Product Requirements Document (PRD)/Sprint 5/PRD_Sprint5_UI_Tree.md
- Product Requirements Document (PRD)/Sprint 5/PRD_Sprint5_Wish_From_Backpack.md
- Product Requirements Document (PRD)/Sprint 6/PRD_Sprint6_Overview.md
- Product Requirements Document (PRD)/Sprint 6/PRD_Sprint6_Security_and_Documentation_Alignment.md
- Product Requirements Document (PRD)/Sprint 7/PRD_Sprint7_Overview.md
- Product Requirements Document (PRD)/Sprint 8/PRD_Sprint8_Admin_Portal_Overview.md

Notes:

- These PRD files correspond to the functional scope and requirement expression of each sprint.
- The consolidated requirements in the main report are the project-level version distilled from these PRD files.

Appendix D. Technical Design and Database Evidence

- Technical Design Document (TDD)/Database Table Structure/familyjoy_db_schema_documentation.md
- Deliverable Document (DVL)/mermaid/familyjoy_problem_domain_sketch.md
- Deliverable Document (DVL)/mermaid/familyjoy_core_use_case_diagram.md
- Deliverable Document (DVL)/mermaid/familyjoy_database_er_diagram.md
- Deliverable Document (DVL)/mermaid/familyjoy_runtime_ssr_flow.md
- Deliverable Document (DVL)/mermaid/familyjoy_route_module_responsibility_map.md
- Deliverable Document (DVL)/mermaid/familyjoy_flow_registration_login.md
- Deliverable Document (DVL)/mermaid/familyjoy_flow_quest.md
- Deliverable Document (DVL)/mermaid/familyjoy_flow_shop.md
- Deliverable Document (DVL)/mermaid/familyjoy_flow_spirit_tree_backpack.md
- Deliverable Document (DVL)/mermaid/familyjoy_flow_calendar.md
- Deliverable Document (DVL)/mermaid/familyjoy_code_structure_overview.md

Notes:

- This group supports the problem domain, use cases, database, runtime architecture, workflow diagrams, and developer-documentation sections.
- familyjoy_db_schema_documentation.md is the single baseline document for database-structure explanation.

Appendix E. Black-Box Testing Evidence

- Black-box Test Document (BTD)/Sprint 1/BBT_Sprint1_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 2/BBT_Sprint2_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 3/BBT_Sprint3_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 4/BBT_Sprint4_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 5/BBT_Sprint5_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 6/BBT_Sprint6_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 7/BBT_Sprint7_Black_Box_Test_Cases.md
- Black-box Test Document (BTD)/Sprint 8/BBT_Sprint8_Black_Box_Test_Cases.md

Notes:

- These BTD documents record the sprint-level manual black-box checks carried out during development. Taken together, they show that validation was performed continuously across feature groups rather than only at the end of the project.
- They support the testing section as lightweight summaries rather than as a commercial QA package.

Appendix F. Screenshots and Figure Assets

Deliverable Document (DVL)/Figures/figure_6_3_planning_documents.jpg

Deliverable Document (DVL)/Figures/figure_6_1_user_story_example.jpg

Primary screenshots for direct role explanation:

- Deliverable Document (DVL)/Screenshots/User_Login.png
- Deliverable Document (DVL)/Screenshots/Parent_Spirit Tree.png
- Deliverable Document (DVL)/Screenshots/Child_Spirit Tree.png
- Deliverable Document (DVL)/Screenshots/Admin_Users.jpg
- Deliverable Document (DVL)/Screenshots/Admin_Familis.jpg
- Deliverable Document (DVL)/Screenshots/Admin_Audit Logs.jpg

Supporting interface screenshots:

- Deliverable Document (DVL)/Screenshots/Parent_Registry.png
- Deliverable Document (DVL)/Screenshots/Parent_Family Management.png
- Deliverable Document (DVL)/Screenshots/Parent_Profile.png
- Deliverable Document (DVL)/Screenshots/Parent_Qeust_Quest Library.png
- Deliverable Document (DVL)/Screenshots/Parent_Qeust_Child Management.png
- Deliverable Document (DVL)/Screenshots/Parent_Shop_Child Management.png
- Deliverable Document (DVL)/Screenshots/Parent_Shop_Wish Warehouse.png
- Deliverable Document (DVL)/Screenshots/Parent_Calendar.png
- Deliverable Document (DVL)/Screenshots/Child_Quests.png
- Deliverable Document (DVL)/Screenshots/Child_Shop.png
- Deliverable Document (DVL)/Screenshots/Child_Backpack.png
- Deliverable Document (DVL)/Screenshots/Child_Calendar.png
- Deliverable Document (DVL)/Screenshots/Child_Mailbox.png

Primary rendered figures:

- Deliverable Document (DVL)/Figures/figure_5_1_problem_domain_sketch.jpg
- Deliverable Document (DVL)/Figures/figure_5_2_core_use_case_diagram.jpg
- Deliverable Document (DVL)/Figures/figure_5_3_admin_use_case_diagram.jpg
- Deliverable Document (DVL)/Figures/figure_10_1_runtime_ssr_flow.jpg
- Deliverable Document (DVL)/Figures/figure_10_2_database_er_diagram.jpg
- Deliverable Document (DVL)/Figures/figure_10_8_quest_workflow.jpg
- Deliverable Document (DVL)/Figures/figure_10_9_shop_workflow.jpg
- Deliverable Document (DVL)/Figures/figure_10_10_spirit_tree_workflow.jpg
- Deliverable Document (DVL)/Figures/figure_11_1_route_module_responsibility_map.jpg
- Deliverable Document (DVL)/Figures/figure_14_1_calendar_flow_validation.jpg

Supporting workflow screenshots:

- Deliverable Document (DVL)/Screenshots/Quest Flow.jpg
- Deliverable Document (DVL)/Screenshots/Shop Flow.jpg
- Deliverable Document (DVL)/Screenshots/Spirit Tree Flow.jpg
- Deliverable Document (DVL)/Screenshots/Calendar Flow.jpg

Notes:

- This group contains the main screenshot and figure assets used directly in the report body, together with closely related supporting images retained for evidence consistency and later verification.
- The Mermaid diagrams used in the report have already been rendered into jpg format for later migration to Word.

Shop Flow.jpg is also used in the main body as direct supporting validation evidence for the testing section.

- The assets are intentionally separated into primary and supporting groups so that the report can stay selective in the main body while still retaining a fuller evidence set in the appendix.

Appendix G. Version Control and Submission Evidence

- Deliverable Document (DVL)/Resource/github_commit_history_evidence.jpg
- Deliverable Document (DVL)/Resource/DLV_FamilyJoy_PRES_v6.pptx
- Deliverable Document (DVL)/Resource/Scrum Image.jpg

Deliverable Document (DVL)/Resource/DongHan WU - Video Clip - MCSD51 - 20260223.mp4

Deliverable Document (DVL)/Resource/github_commit_history_full.md

Notes:

- github_commit_history_evidence.jpg provides a visible repository-history snapshot, while github_commit_history_full.md preserves the fuller main-branch commit history used for version-control evidence. Report-branch commits are intentionally excluded from this export.
- The presentation video clip is included as a supporting submission file under the filename DongHan WU - Video Clip - MCSD51 - 20260223.mp4.